

TEORÍA (8 puntos)

Apellidos y nombre:

Pregunta 1. ¿Qué es una «inspección Fagan»?

Pregunta 2. Indica dos desventajas de la técnica de depuración de errores basada en utilizar chivatos

Pregunta 3. Indica el significado de las siguientes etiquetas para métodos utilizadas en JUnit4:

@Test:

@Before:

@After:

@BeforeClass:

@AfterClass:

Pregunta 4. ¿Por qué es relevante la Interfaz de Usuario en las pruebas de portabilidad?

Pregunta 5. Enumera tres tipos distintos de objetivos de pruebas que se pueden utilizar como punto de partida para realizar el análisis de riesgos de un producto y justifica su interés.

Pregunta 6. Describe brevemente tres herramientas que permitan enviar eventos sobre el interfaz de usuario de aplicaciones desarrolladas para dispositivos Android.

Pregunta 7. ¿Cuáles son las fases del ciclo de vida de TMAP para un nivel de pruebas de sistema?

Pregunta 8. Describe brevemente en qué consiste la técnica de estimación basada en ratios y para qué se utiliza dentro de la actividad de estimación del esfuerzo del proceso de pruebas.

Pregunta 9. Describe el tipo de cobertura y la información requerida en la base de las pruebas para aplicar la técnica de prueba de caminos con nivel de profundidad 2.

Pregunta 10. Describe brevemente la actividad de “determinar la estrategia de pruebas” dentro de la fase de planificación de la gestión global del proceso de pruebas.

EJERCICIO (2 puntos)

Diseña pruebas para el siguiente código utilizando la técnica de pruebas de caminos con nivel de profundidad igual a 3:

1. Obtén primero el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/expresiones y nodos que se indica en el propio código (0,2 puntos).
2. Aplica la técnica de pruebas de caminos con profundidad 3 para obtener el conjunto de caminos completos correspondiente (0,9 puntos).
3. Finalmente, establece casos de prueba para los caminos obtenidos, incluyendo entradas y salidas del método (0,9 puntos).

```
/**
 * Compara los contenidos de los vectores «a» y «b» y devuelve true si y
 * solo si son iguales, es decir, si tienen el mismo número de
 * componentes y cada i-ésima componente de «a» es igual a la de «b».
 *
 * @param a - un vector no nulo.
 * @param b - otro vector no nulo.
 * @return true si y solo si los vectores «a» y «b» son iguales.
 */
public static boolean arrayEquals(int[] a, int[] b) {
    A boolean iguales = false;
    if (B a.length == b.length) {
        C int i = 0;
        while (D i < a.length && E a[i] == b[i]) {
            F i++;
        }
        G iguales = i == a.length;
    }
    H return iguales;
}
```